

Lab 3 – Component Modelling & Architectural Pattern Selection

Architecture Selection

We chose Layered Architecture for the Self-Service Coffee Kiosk System.

Architectural Choice

Layered Architecture is the most suitable choice because the coffee kiosk is a relatively small system with clearly separated responsibilities. The system can be organized into presentation, business logic, service, and data/hardware layers.

Reason 1 – Clear Separation of Responsibilities

The architecture separates the touch-screen interface from the core ordering and payment logic. The Kiosk UI handles customer interactions, while the Order Manager handles order processing and the Payment Service handles credit-card payments. This makes the system easier to understand, maintain, and modify.

Reason 2 – Simplicity and Maintainability

The kiosk has a limited number of functions and does not require independently deployable services. Using microservices would introduce unnecessary network communication and operational complexity. A layered design provides a simpler solution while still allowing individual components such as the payment service or receipt printer to be replaced.

Security Advantage

Security-sensitive payment processing can be isolated within the Payment Service component. The Kiosk UI does not need to directly handle the internal payment-processing logic, reducing unnecessary exposure of payment-related operations and allowing access between components to be controlled through defined interfaces.

Performance Benefit

Because the components are part of a local kiosk system, communication between the UI, Order Manager, Menu & Pricing Manager, and other components can occur with low latency. The architecture avoids the additional network overhead that would normally be introduced by a distributed microservices architecture.

Conclusion

Layered Architecture provides a simple, maintainable, and efficient structure for the Self-Service Coffee Kiosk. It clearly separates user interaction, business logic, payment processing, data access, and hardware interaction while supporting the required functionality.